

رشته‌ها و Template Literals (۱)

```
const user = {
  name: "سارا",
  age: 28,
  city: "تهران",
  isPremium: true
};

// با روش قدیمی (دردسر!)
const messageOld = "سلام " + user.name + "!\n" +
  "سن شما: " + user.age + "\n" +
  "شهر: " + user.city + "\n" +
  ("معمولی": "پریمیوم" ? user.isPremium ? "وضعیت عضویت");

// ساده و زیبا (Template Literals با)
const messageNew = `👋 سلام ${user.name}!
🕒 سن شما: ${user.age}
🏠 شهر: ${user.city}
⭐ وضعیت عضویت: ${user.isPremium ? "پریمیوم 🏆" : "معمولی"}
💰 ${user.isPremium ? "به پریمیوم پیوندید" : "تخفیف دارید 50%"}`;

console.log(messageNew);
```

۲) آرایه‌ها و متدهای مهم

```
// ایجاد آرایه
const numbers = [1, 2, 3, 4, 5];
const users = ["علی", "مریم", "رضا"];

// تبدیل آرایه - map متد
const doubled = numbers.map(num => num * 2);
// [2, 4, 6, 8, 10]
const userList = users.map((user, index) => ({
  id: index + 1,
  name: user,
  email: `${user}@example.com`
}));

// فیلتر کردن - filter متد
const evenNumbers = numbers.filter(num => num % 2 === 0);
// [2, 4]

const longNames = users.filter(user => user.length > 3);
// ["مریم"]

// مثال ترکیبی
const products = [
  { id: 1, name: "لپتاپ", price: 15000000, category: "الکترونیک" },
  { id: 2, name: "کتاب", price: 50000, category: "آموزشی" },
  { id: 3, name: "هدفون", price: 800000, category: "الکترونیک" }
];

// محصولات الکترونیکی با قیمت بالای ۱ میلیون
const expensiveElectronics = products
  .filter(product => product.category === "الکترونیک")
  .filter(product => product.price > 1000000)
  .map(product => `${product.name}: ${product.price.toLocaleString()} تومان`);
```

۳) اشیاء ساده (Literal Object)

```
// ایجاد شیء
const student = {
  // ویژگی‌ها (Properties)
  firstName: "سارا",
  lastName: "محمدی",
  age: 20,
  courses: ["ریاضی", "فیزیک", "برنامه‌نویسی"],

  // متدها (Methods)
  getFullName() {
    return `${this.firstName} ${this.lastName}`;
  },

  addCourse(course) {
    this.courses.push(course);
    return this.courses;
  },

  getInfo() {
    return `
      نام: ${this.getFullName()}
      سن: ${this.age}
      تعداد دروسها: ${this.courses.length}
    `;
  }
};

// دسترسی به ویژگی‌ها
console.log(student.firstName); // "سارا"
console.log(student["lastName"]); // "محمدی"

// فراخوانی متدها
console.log(student.getFullName()); // "سارا محمدی"
student.addCourse("زبان انگلیسی");

// Destructuring
const { firstName, age } = student;
console.log(firstName, age); // "سارا" "20"

// Spread Operator
const studentCopy = { ...student, id: 12345 };
```

۴) Fetch API (روش مدرن از سال ۲۰۱۵ - مبتنی بر Promise)

<pre>fetch('https://jsonplaceholder.typicode.com/users') .then(response => response.json()) .then(users => { users.forEach(user => { console.log(user.name); }); }); fetch('https://api.example.com/save', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ name: 'علی', age: ۲۵ }) });</pre>	مزایای Fetch  ۱. سینتکس ساده و خوانا ۲. بر پایه Promise (مدیریت آسان) ۳. Integration با async/await معایب Fetch  ۱. پشتیبانی در IE قدیمی نیست ۲. پیشرفت upload سخت‌تر ۳. خطاهای شبکه را reject نمی‌کند (فقط ۴۰۴، ۵۰۰ و ...)
--	---

زنجیره then چیه؟

یه روش برای پشت سر هم گذاشتن کارها وقتی با Promise کار می‌کنیم.
مثل این می‌مونه که بگی: "اول این کار رو بکن، بعد اون کار رو، بعد اون یکی رو ..."

۵) XMLHttpRequest (روش قدیمی از سال ۲۰۰۶ - مبتنی بر Event)

<pre>const xhr = new XMLHttpRequest(); xhr.open('GET', 'https://api.example.com/data'); xhr.onload = function() { if (xhr.status === ۲۰۰) { const data = JSON.parse(xhr.responseText); console.log(data); } }; xhr.send();</pre>	مزایای XMLHttpRequest  ۴. پشتیبانی کامل از همه مرورگرها ۵. ردیابی پیشرفت upload/download ۶. لغو درخواست با xhr.abort() ۷. Timeout تنظیم زمان انتظار معایب XMLHttpRequest  ۱. کدنویسی طولانی و پیچیده ۲. Callback Hell (درهم‌ریختگی) ۳. ساختار قدیمی
--	--